Chapitre 4: Complexité algorithmique

Introduction

Le temps d'exécution dépend de :

- Les données du programme
- Le compilateur / langage
- La machine (vitesse, mémoire)
- La complexité de l'algorithme

Objectif

Mesurer la complexité indépendamment de la machine et du langage, uniquement en fonction de la taille n des données à traiter.

Exemple : calculer $P(x) = a_n x^n + \ldots + a_1 x + a_0$

Variante 1 : naïve

```
P ← T[0]
Pour i ← 1 à n faire
  P ← P + T[i] * Puiss(X,i)
```

Coût

n additions
n multiplications
n appels à Puiss

Variante 2 : Inter

```
Inter ← X ; P ← T[0]
Pour i ← 1 à n faire
  P ← P + Inter * T[i]
  Inter ← Inter * X
```

Coût

n additions
2n multiplications
4 variables

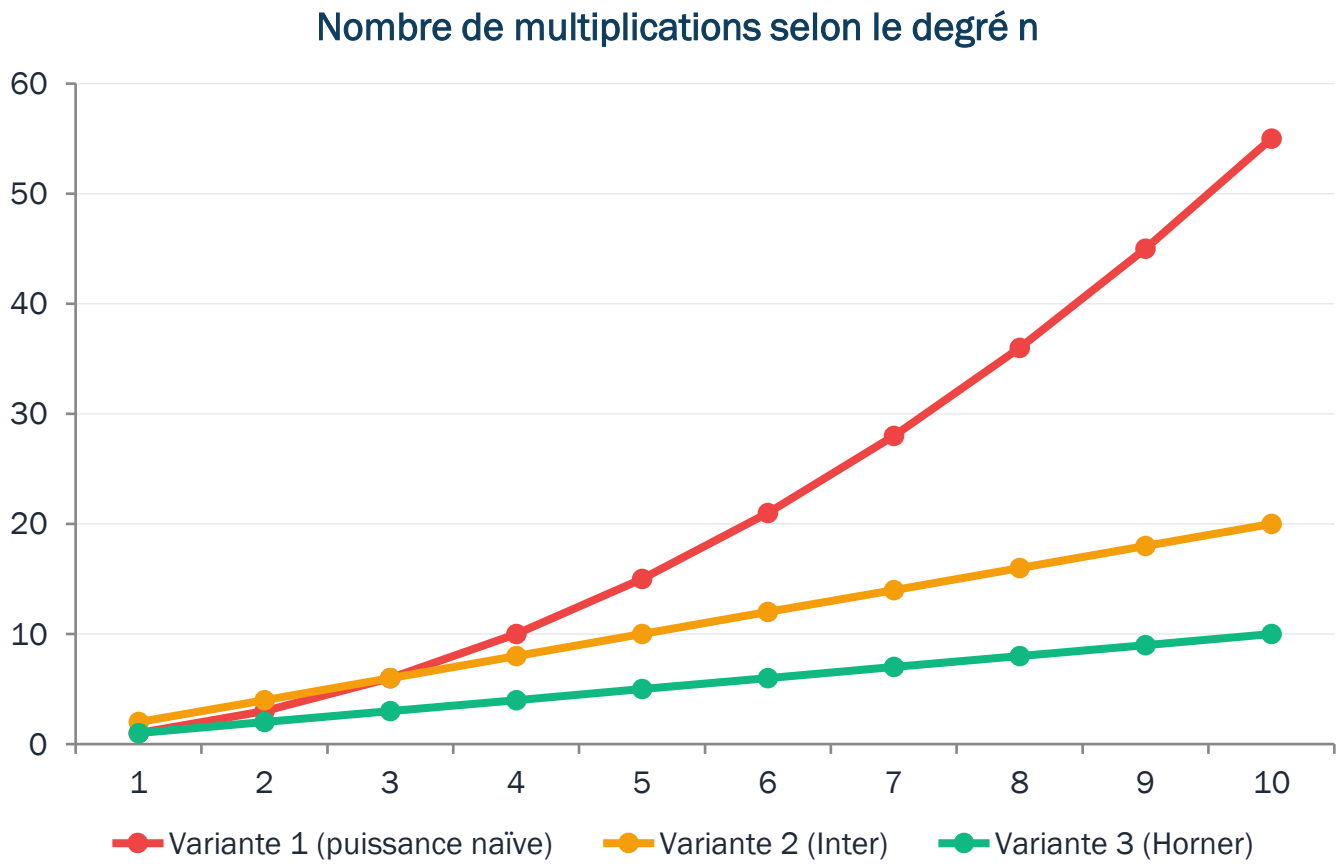
Variante 3 : Horner

```
P ← T[n]
Pour i ← n-1 à 0 faire
  P ← P * X + T[i]
```

Coût

n additions
n multiplications
3 variables

Exemple : calculer $P(x) = a_n x^n + \ldots + a_1 x + a_0$



	V1	V2	V3
Additions	n	n	n
Multipl.	$n^2/2$	2n	n
Variables	P,X,i+Puiss	P,X,i,Inter	P,X,i

À retenir

Estimer la complexité en temps ET en espace avant d'écrire et d'implémenter l'algorithme.

Définition

Définition:

La complexité (temporelle) d'un algorithme est le nombre d'opérations fondamentales (affectations, comparaisons, opérations arithmétiques) effectuées sur un jeu de données, exprimé comme une fonction de la taille **n** de ce jeu. Elle permet de comparer deux algorithmes indépendamment de la machine, du langage et du compilateur.

Tri à Bulles

```
Algorithme: TriBulles(T,n)
pour i = 1 … n-1 faire
    pour j = n … i+1 faire
      si T[j] < T[j-1] alors
        échanger T[j], T[j-1]
      fin
    fin
fin
```

Complexité en comparaisons

comparaison $T[j] < T[j-1]$

$\mathbf{n(n-1)/2}$ fois

Tri à Bulles

```
Algorithme: TriBulles(T,n)
pour i = 1 … n-1 faire
    pour j = n … i+1 faire
      si T[j] < T[j-1] alors
        échanger T[j], T[j-1]
      fin
    fin
fin
```

Complexité en échange

Echanger T[j] et T[j−1]

- **0** fois au mieux,
- **n(n-1)/2** au pire

Tri à Bulles

Au mieux

0

Tableau déjà trié : aucun couple dans le mauvais ordre.

En moyenne

$n(n-1)/4$

La moitié des couples sont des inversions.

Au pire

$n(n-1)/2$

Tableau trié à l'envers : tous les couples sont inversés.

Le nombre total d'échanges est le nombre de couples d'indices (i, j) où i < j dont les valeurs dans T ne sont pas dans le bon ordre (i.e T[i] > T[j]).

Définition

Définition:

Soit $C_A(d)$ la complexité en temps de l'algorithme A sur la donnée **d** et D_n l'ensemble des données de taille **n**. On appelle complexité :

- ❖ dans le meilleur des cas : $\min_A(n) = \min\{c_A(d) \mid d \in Dn\}$
- ❖ dans le pire des cas : $\max_A(n) = \max\{c_A(d) \mid d \in Dn\}$
- ❖ en moyenne : $moy_A(n) = \sum_{d \in D_n} prob(d) c_A(d)$

où prob(d) est la probabilité que l'on ait l'instance d.

Tri rapide

```
Algorithme: Partitionnement(T,a,b,m)
l = a + 1 ;
k = b ;
tant que l ≤ k faire
  tant que T[l] ≤ T[a] faire
     l = l + 1 ;
  fin
  tant que T[k] > T[a] faire
     k = k - 1 ;
  fin
  si (l < k) alors
     echanger(T[l], T[k]) ;
     l = l + 1 ;
     k = k - 1 ;
  fin
fin
m = k ;
echanger(T[a], T[k]) ;
fin
```

Nombre de comparaisons

Au pire

$((n+1)(n+2)/2) - 3$

T est déjà trié

Tri rapide

```
Algorithme: Partitionnement(T,a,b,m)
l = a + 1 ;
k = b ;
tant que l ≤ k faire
  tant que T[l] ≤ T[a] faire
     l = l + 1 ;
  fin
  tant que T[k] > T[a] faire
     k = k - 1 ;
  fin
  si (l < k) alors
     echanger(T[l], T[k]) ;
     l = l + 1 ;
     k = k - 1 ;
  fin
fin
m = k ;
echanger(T[a], T[k]) ;
fin
```

Nombre de comparaisons

Au mieux

$(n+1)(\log_2(n+1)-1)$

T divisé en 2 moitiés égales à chaque itération

Comparaison des tris

Tri	Au mieux	Au pire	En moyenne
Bulles	0	$n(n-1)/2$	$n(n-1)/4$
Rapide	$(n+1)(\log_2(n+1) -1)$	$((n+1)(n+2)/2) -3$	$\Theta(n \log_2 n)$

Notation asymptotique Θ

Définition:

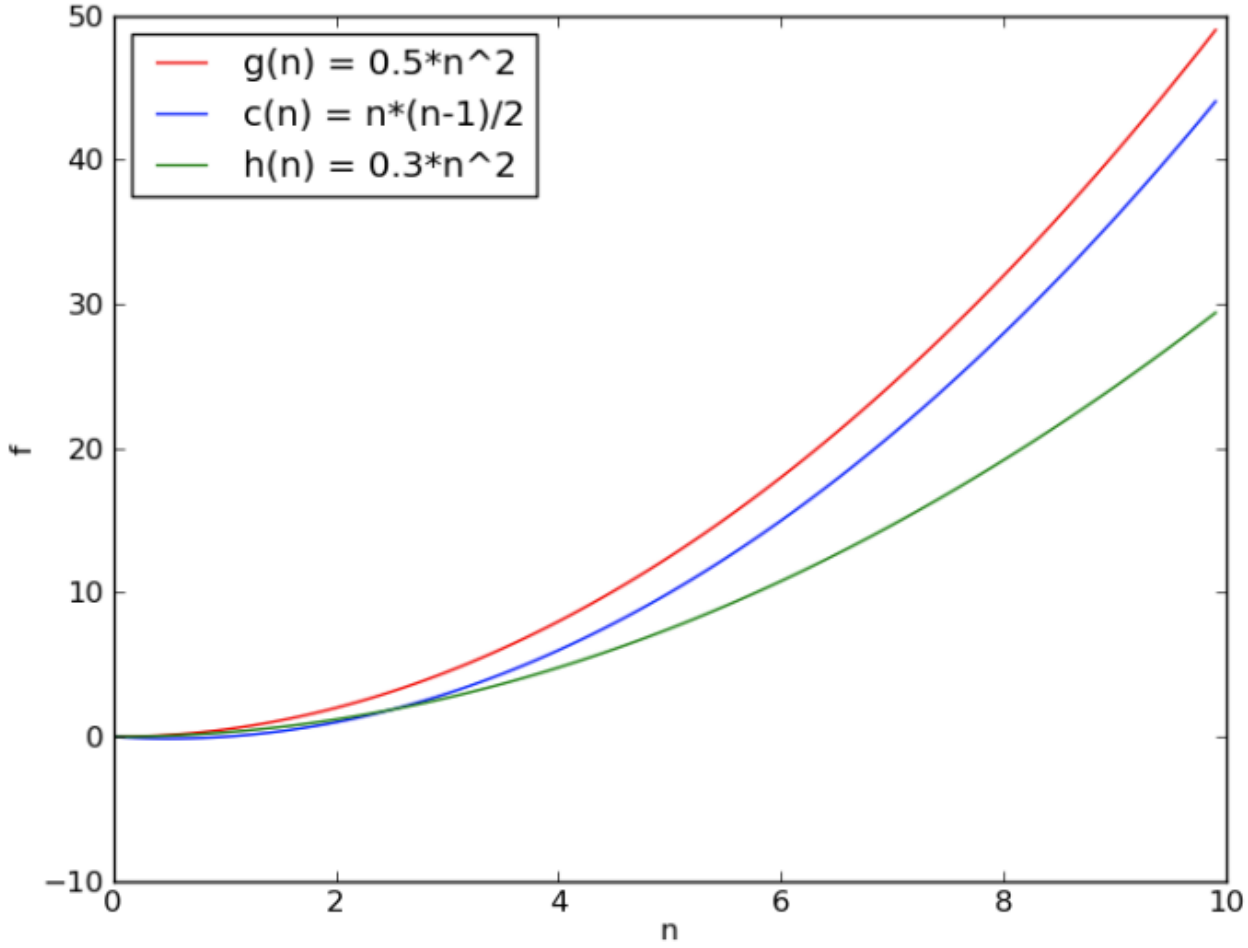
Soit **C(n)** une fonction exprimant le temps d'exécution d'un algorithme pour des données de taille **n**. **C(n)** et **g(n)** ont même ordre de grandeur asymptotique ssi, à partir d'un certain rang **N**, on a l'encadrement :

$$cg(n) \leq C(n) \leq dg(n)$$

où **c** et **d** sont des constantes positives.

Notation asymptotique Θ

Exemple: $\frac{n(n-1)}{2} \in \Theta(n^2)$



Notation (mathématique) asymptotique Θ

L'ensemble des fonctions qui ont même ordre de grandeur que $g(n)$ se note $\Theta(g(n))$.

Définition:

Soit $g: \mathbb{N} \to \mathbb{N}$. On note

$$\Theta\big(g(n)\big) = \{f: \mathbb{N} \to \mathbb{N} \mid \exists\, c, d > 0,\ \exists\, N \in \mathbb{N},\ cg(n) \leq f(n) \leq dg(n),\ \forall\, n \geq N\}.$$

On note $\mathrm{C(n)} \in \Theta(g(n))$.

Notation asymptotique O

Définition:

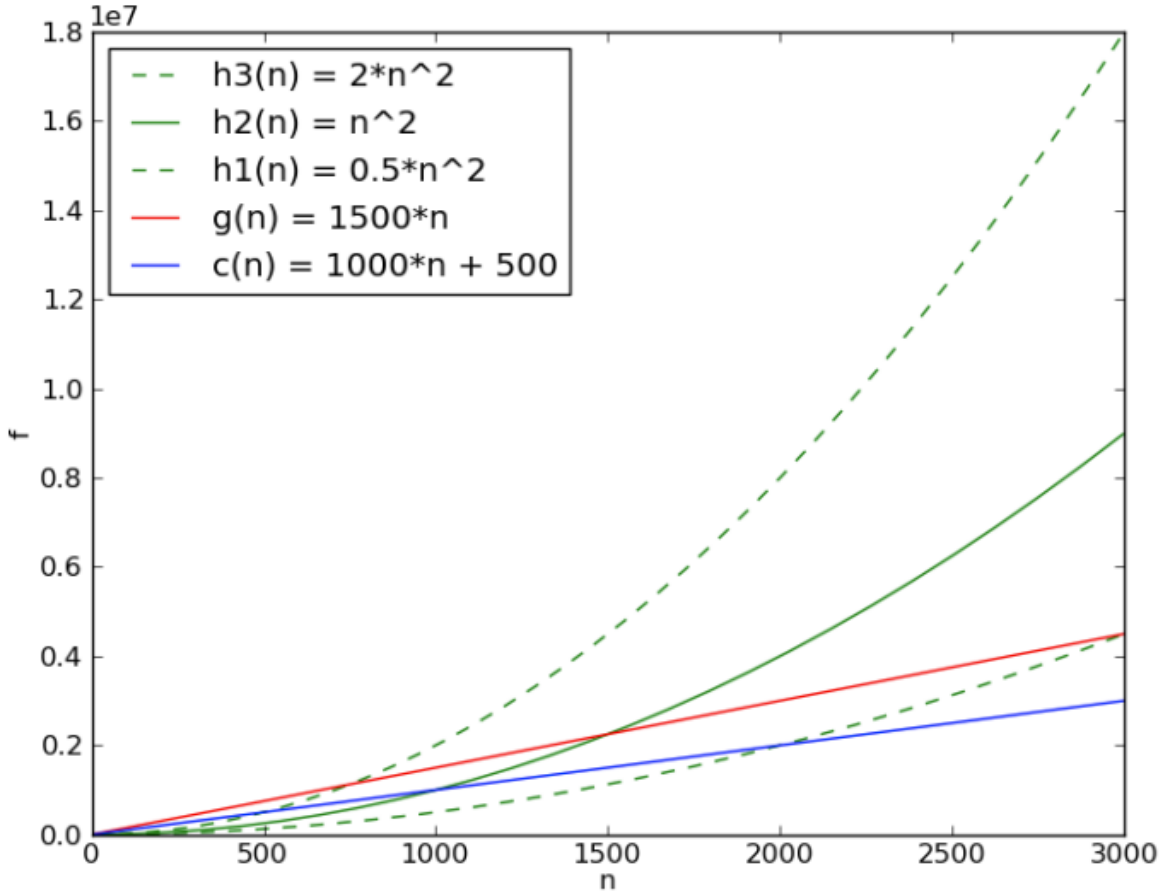
Soit $g: \mathbb{N} \to \mathbb{N}$. On note

$$O\big(g(n)\big) = \{f: \mathbb{N} \to \mathbb{N} \mid \exists\, c > 0, \exists\, N \in \mathbb{N},\ f(n) \leq cg(n), \forall\, n \geq N\}.$$

On note $\mathrm{C}(\mathrm{n}) \in O\big(g(n)\big)$, on dit que C est dominée asymptotiquement par g.

On note aussi $\mathrm{C} = O(g)$

Notation asymptotique O



Notation asymptotique O

Définition:

Soit $g: \mathbb{N} \to \mathbb{N}$. On note

$$\Omega\big(g(n)\big) = \{f: \mathbb{N} \to \mathbb{N} \mid \exists\, c > 0, \exists\, N \in \mathbb{N}, cg(n) \leq f(n) \ \forall\, n \geq N\}.$$

Si $C(n) \in \Omega\big(g(n)\big)$, on dit que C domine asymptotiquement g.

On note aussi $C = \Omega(g)$

Exemple: $n^3 \in \Omega(n^2)$

Notation asymptotique

Θ(g) : même ordre	O(g) : borne sup	Ω(g) : borne inf
$c \cdot g \leq f \leq d \cdot g$	$f \leq c \cdot g$	$c \cdot g \leq f$
$\exists\, c, d > 0, \exists\, N, \forall\, n \geq N.$	$\exists\, c > 0, \exists\, N, \forall\, n \geq N.$	$\exists\, c > 0, \exists\, N, \forall\, n \geq N.$
Ex : $n(n-1)/2 \in \Theta(n^2)$	**Ex : $n(n-1)/2 \in O(n^2)$**	**Ex : $n^3 \in \Omega(n^2)$**

$\Theta(g) = O(g) \cap \Omega(g)$: f est encadrée par g des deux côtés.

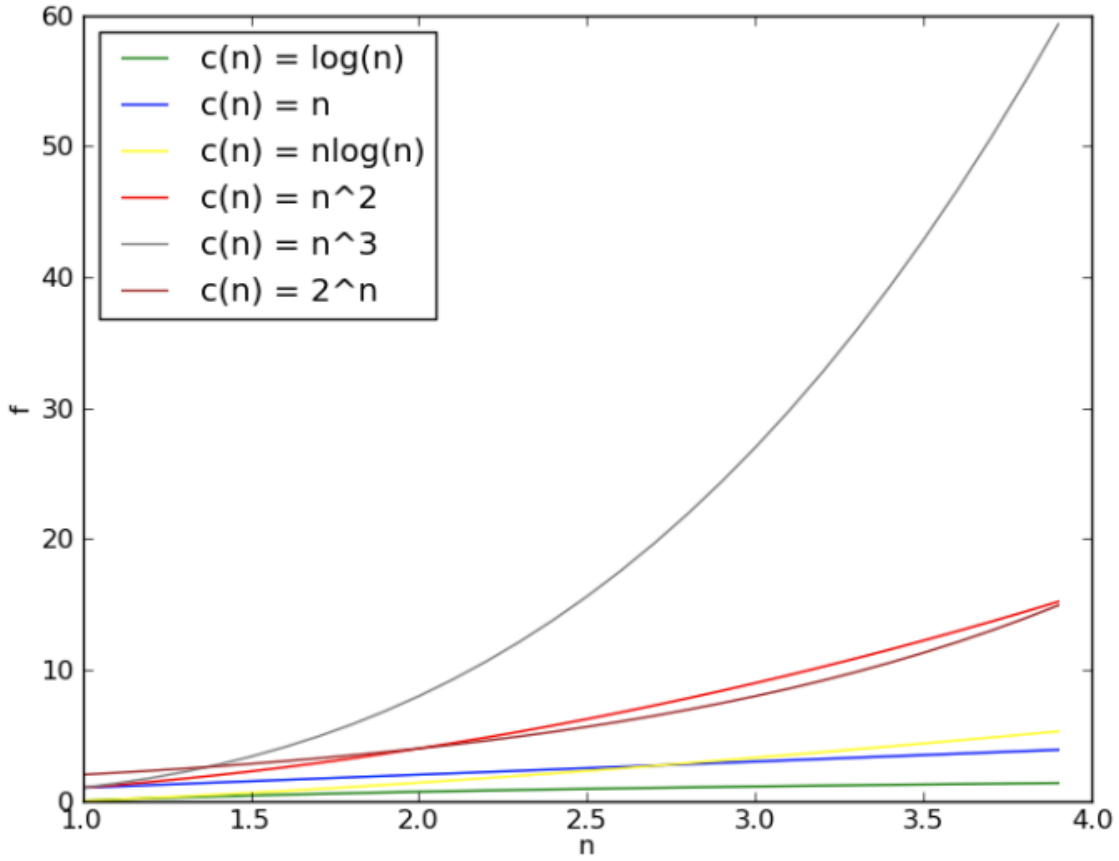
Complexité de tris

Tri	Au mieux	Au pire	En moyenne
Bulles	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Rapide	$\Theta(n \log_2 n)$	$\Theta(n^2)$	$\Theta(n \log_2 n)$
Tas	$O(n \log_2 n)$	$\Theta(n \log_2 n)$	$\Theta(n \log_2 n)$

Echelle de comparaison

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

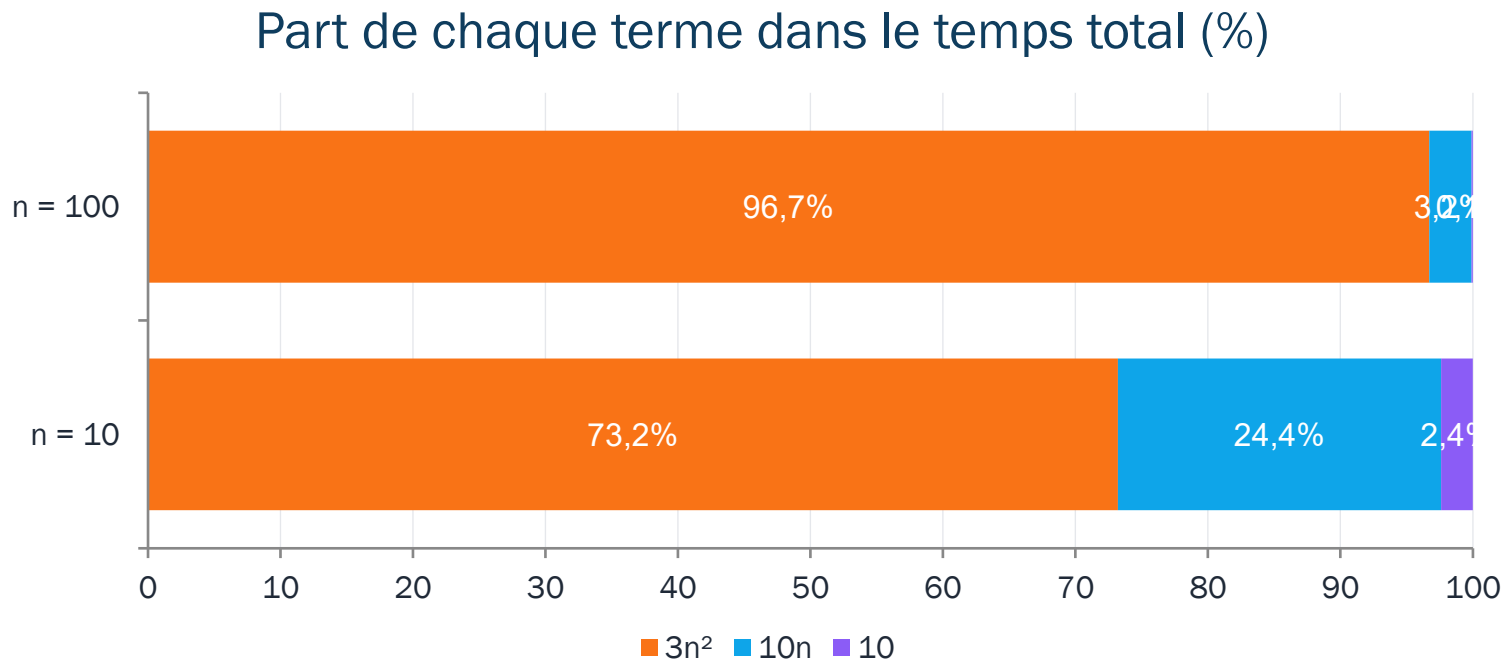
constante · logarithmique · linéaire · quasi-linéaire · quadratique · cubique · exponentielle · factorielle



Notation asymptotique

$T(n) = 3n^2 + 10n + 10$

$O(T(n)) = O(\max(3n^2, 10n, 10)) = O(3n^2) = O(n^2)$



Dès que n grandit, le terme dominant ($3n^2$) pèse presque tout : on garde n^2 et on oublie le reste.

Quelques chiffres

Machine à 1 opération / seconde

Opérations	Durée
10^2	1,7 min
10^4	2,8 h
10^6	1,6 semaine
10^7	3,8 mois
10^8	3,1 ans
10^9	3,1 décennies
10^{10}	3,1 siècles

Pour n = 10^6 éléments

Op./s	n	n log n	n^2
10^6	1 s	20 s	12 j
10^9	instantané	instantané	17 min
10^{12}	instantané	instantané	1 s

Un algorithme en n^2 devient inutilisable bien avant un n log n : améliorer l'algorithme vaut mieux qu'acheter une machine plus rapide.